

AI Engineer Case: Document Intelligence Pipeline for a Legal Analytics

Overview

You are assisting a legal firm in automating the ingestion and analysis of diverse documents, including invoices, contracts, meeting minutes, and emails. These documents come in various formats such as PDFs (which may contain text, images, tables, and figures), Word and Excel files, and images. The primary objective is to extract metadata and key information for subsequent analytics and retrieval.

Specifically, the firm aims to conduct a proof of concept (POC) to digitize their operations within their “Claims” line of business, where significant value can be derived from automation. The scope of this initiative includes:

- Developing data pipelines to ingest, classify, and extract entities such as client names, invoice amounts, dates, and involved parties from documents.
- Implementing data storage and modelling solutions to facilitate downstream querying and maintain audit trails.
- Ensuring extensibility for future integration of large language model (LLM)-based summarization or question-answering capabilities.

Tasks

1. Present your technical approach through two slides.
 - a. **High-Level Overview:** Summarize the problem, your proposed approach, and your chosen technical stack. Your slide should reflect a platform view that addresses how the pipeline handles failures at each stage, how you would scale to processing large document volumes, and how your data model supports the audit trail requirement described above. These do not need to be exhaustive, a sentence or a diagram annotation per point is sufficient.
 - b. **Detailed Execution Pipeline:** Illustrate a step-by-step view of how your pipeline ingests, classifies, and processes incoming documents and stores the results for downstream use cases including reporting, search, summarization, and aggregation.
2. Implement a document extraction service in Python that processes PDF invoices and extracts header and line-item data into a structured format using a modern LLM/VLM approach. The service should be structured as a proper backend component with clear separation between document handling, LLM interaction, and output validation, i.e., not a standalone script or notebook. Include structured logging at key processing steps. Your service must explicitly handle the following:
 - a. LLM output that cannot be parsed into the expected schema
 - b. PDFs that contain no embedded text layer (image-only scans)



- c. Transient LLM API failures
- 3. Expose your extraction service via an API designed for real-world document processing latency. The endpoint should accept a PDF document and return a job identifier immediately rather than blocking until extraction is complete. Implement a second endpoint to retrieve the job status and result once processing is finished

Note

We are not looking for a production-complete implementation. We are looking for the logic and judgment in how you structure the service and handle cases where things go wrong.

